**VESC ASS 9**

**NAME:**Esha Sachin Zagade
**ROLL NO :**EN23107138

# Step 1: Install Required Tools

## Install Docker:

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
docker --version
```

## Install Kubernetes CLI:

```
sudo snap install kubectl --classic
kubectl version --client
```

## Install Minikube:

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
```

# Step 2: Start Minikube Cluster

```
minikube start --driver=docker
kubectl get nodes
```

- Output should show minikube Ready

# Step 3: Create Project Folder & AI App

```
mkdir ai-k8-app
cd ai-k8-app
```

## Create app.py:

```
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.route("/")Step 1: Install Required Tools
Install Docker:
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
docker --version
Install Kubernetes CLI:
sudo snap install kubectl --classic
kubectl version --client
Install Minikube:
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
```

```
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
```

Step 2: Start Minikube Cluster

```
minikube start --driver=docker
kubectl get nodes
```

Output should show minikube Ready

Step 3: Create Project Folder & AI App

```
mkdir ai-k8-app
cd ai-k8-app
```

Create app.py:

```
from flask import Flask, request, jsonify
app = Flask(__name__)

@app.route("/")
def home():
    return "AI Model Running in Kubernetes"

@app.route("/predict", methods=["POST"])
def predict():
    data = request.json["number"]
    result = data * 2
    return jsonify({"result": result})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Create requirements.txt:

```
Flask
```

Step 4: Create Dockerfile

```
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install flask
EXPOSE 5000
CMD ["python","app.py"]
```

Step 5: Build Docker Image

```
sudo docker build -t ai-model .
docker images
```

Step 6: Load Image into Minikube

```
minikube image load ai-model
minikube image ls | grep ai-model
```

Output should show docker.io/library/ai-model:latest

Step 7: Create Kubernetes Deployment

Create deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ai-app
  template:
```

```
  metadata:
    labels:
      app: ai-app
  spec:
    containers:
    - name: ai-container
      image: ai-model
      imagePullPolicy: Never
      ports:
      - containerPort: 5000
      resources:
        requests:
          cpu: "100m"
        limits:
          cpu: "500m"
```

Deploy it:

```
kubectl apply -f deployment.yaml
kubectl get pods
```

Step 8 : Create Kubernetes Service

Create service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: ai-service
spec:
  type: NodePort
  selector:
    app: ai-app
  ports:
  - port: 5000
    targetPort: 5000
    nodePort: 30007
```

Deploy service:

```
kubectl apply -f service.yaml
kubectl get svc
```

Access app:

```
minikube service ai-service
```

Browser should show: AI Model Running in Kubernetes

Step 9 : Enable Metrics Server

```
minikube addons enable metrics-server
kubectl get pods -n kube-system
# Wait for metrics-server pod to show 1/1 Running
kubectl top pods
kubectl top nodes
```

Step 10 : Create Horizontal Pod Autoscaler (HPA)

```
kubectl autoscale deployment ai-app --cpu-percent=50 --min=2 --max=10
kubectl get hpa
kubectl describe hpa ai-app
```

Current CPU usage and replicas will appear

Step 1 1 : Generate Load (to trigger scaling)
while true; do curl $(minikube service ai-service --url); done

In another terminal:

kubectl get pods -w

Pods will increase when CPU crosses threshold → proves auto-scaling works

Step 1 2 : Monitor Health & Performance
kubectl top pods
kubectl top nodes
kubectl describe pod <pod-name>
kubectl logs <pod-name>
Step 1 3 : Open Kubernetes Dashboard (Optional but Recommended)
minikube dashboard

Opens a web dashboard showing pods, deployments, services, metrics, auto-scaling

Step 1 4 : Final Commands for Exam Output
kubectl get pods
kubectl get svc
kubectl get hpa
kubectl top pods
kubectl top nodes
kubectl describe hpa ai-app

```
def home():
    return "AI Model Running in Kubernetes"
@app.route("/predict", methods=["POST"])
def predict():
    data = request.json["number"]
    result = data * 2
    return jsonify({"result": result})
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

## Create requirements.txt:

```
Flask
```

# Step 4 : Create Dockerfile

```
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install flask
EXPOSE 5000
CMD ["python","app.py"]
```

# Step 5 : Build Docker Image

sudo docker build -t ai-model .
docker images

# Step 6: Load Image into Minikube

```
minikube image load ai-model
minikube image ls | grep ai-model
```

- Output should show docker.io/library/ai-model:latest

# Step 7: Create Kubernetes Deployment

Create deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ai-app
  template:
    metadata:
      labels:
        app: ai-app
    spec:
      containers:
      - name: ai-container
        image: ai-model
        imagePullPolicy: Never
        ports:
        - containerPort: 5000
        resources:
          requests:
            cpu: "100m"
          limits:
            cpu: "500m"
```

Deploy it:

```
kubectl apply -f deployment.yaml
kubectl get pods
```

# Step 8: Create Kubernetes Service

Create service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: ai-service
spec:
  type: NodePort
  selector:
    app: ai-app
  ports:
  - port: 5000
    targetPort: 5000
    nodePort: 30007
```

Deploy service:

```
kubectl apply -f service.yaml
kubectl get svc
```

Access app:

```
minikube service ai-service
```

- Browser should show: AI Model Running in Kubernetes

## Step 9: Enable Metrics Server

```
minikube addons enable metrics-server
kubectl get pods -n kube-system
# Wait for metrics-server pod to show 1/1 Running
kubectl top pods
kubectl top nodes
```

## Step 10: Create Horizontal Pod Autoscaler (HPA)

```
kubectl autoscale deployment ai-app --cpu-percent=50 --min=2 --max=10
kubectl get hpa
kubectl describe hpa ai-app
```

- Current CPU usage and replicas will appear

## Step 11: Generate Load (to trigger scaling)

```
while true; do curl $(minikube service ai-service --url); done
```

In another terminal:

```
kubectl get pods -w
```

- Pods will increase when CPU crosses threshold → proves auto-scaling works

## Step 12: Monitor Health & Performance

```
kubectl top pods
kubectl top nodes
kubectl describe pod <pod-name>
kubectl logs <pod-name>
```

## Step 13: Open Kubernetes Dashboard (Optional but Recommended)

```
minikube dashboard
```

- Opens a web dashboard showing pods, deployments, services, metrics, auto-scaling

## Step 14: Final Commands for Exam Output

```
kubectl get pods
```

kubectl get svc
kubectl get hpa
kubectl top pods
kubectl top nodes
kubectl describe hpa ai-app

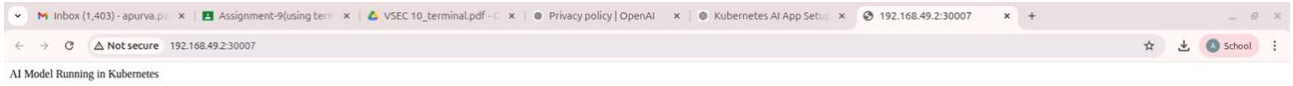
```
admin1@bflcomp20: ~/ai-k8-app
app.py  Dockerfile  requirements.txt
(base) admin1@bflcomp20:~/ai-k8-app$ nano deployment.yaml
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl apply -f deployment.yaml
deployment.apps/ai-app configured
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl get pods
NAME                      READY   STATUS        RESTARTS   AGE
ai-app-5c86c57794-4mc82   1/1     Running       0          17s
ai-app-5c86c57794-c8xxg   1/1     Running       0          16s
ai-app-789d94fc58-b9b8l   1/1     Terminating   0          17m
ai-app-789d94fc58-xcdlk   1/1     Terminating   0          17m
(base) admin1@bflcomp20:~/ai-k8-app$ nano service.yaml
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl apply -f service.yaml
service/ai-service configured
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl get svc
NAME         TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
ai-service   NodePort    10.111.5.210   <none>        5000:30007/TCP   16m
kubernetes   ClusterIP   10.96.0.1      <none>        443/TCP          21m
(base) admin1@bflcomp20:~/ai-k8-app$ minikube service ai-service
|  NAMESPACE  |  NAME  |  TARGET PORT  |  URL  |
|  default  |  ai-service  |  5000  |  http://192.168.49.2:30007  |
🎉  Opening service default/ai-service in default browser...
(base) admin1@bflcomp20:~/ai-k8-app$ Opening in existing browser session.
[26213:26213:0100/000000.450605:ERROR:content/zygote/zygote_linux.cc:672] write: Broken pipe (32)
(base) admin1@bflcomp20:~/ai-k8-app$ minikube service ai-service
|  NAMESPACE  |  NAME  |  TARGET PORT  |  URL  |
|  default  |  ai-service  |  5000  |  http://192.168.49.2:30007  |
🎉  Opening service default/ai-service in default browser...
(base) admin1@bflcomp20:~/ai-k8-app$ Opening in existing browser session.
(base) admin1@bflcomp20:~/ai-k8-app$ minikube addons enable metrics-server
💡  metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
    ▪ Using image registry.k8s.io/metrics-server/metrics-server:v0.8.1
🌟  The 'metrics-server' addon is enabled
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl top pods
error: Metrics API not available
(base) admin1@bflcomp20:~/ai-k8-app$ minikube addons enable metrics-server
💡  metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
```

```
admin1@bflcomp20: ~/ai-k8-app
error: Metrics API not available
(base) admin1@bflcomp20:~/ai-k8-app$ minikube addons enable metrics-server
💡  metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
    ▪ Using image registry.k8s.io/metrics-server/metrics-server:v0.8.1
🌟  The 'metrics-server' addon is enabled
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl get pods -n kube-system
NAME                                   READY   STATUS    RESTARTS      AGE
coredns-7d764666f9-q9tpf               1/1     Running   0             24m
etcd-minikube                          1/1     Running   0             24m
kube-apiserver-minikube                1/1     Running   0             24m
kube-controller-manager-minikube       1/1     Running   0             24m
kube-proxy-ncnwf                       1/1     Running   0             24m
kube-scheduler-minikube                1/1     Running   0             24m
metrics-server-9d74bb658-s5shx         0/1     Running   0             40s
storage-provisioner                    1/1     Running   1 (24m ago)   24m
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl top pods
error: Metrics API not available
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl get pods -n kube-system
NAME                                   READY   STATUS    RESTARTS      AGE
coredns-7d764666f9-q9tpf               1/1     Running   0             25m
etcd-minikube                          1/1     Running   0             25m
kube-apiserver-minikube                1/1     Running   0             25m
kube-controller-manager-minikube       1/1     Running   0             25m
kube-proxy-ncnwf                       1/1     Running   0             25m
kube-scheduler-minikube                1/1     Running   0             25m
metrics-server-9d74bb658-s5shx         1/1     Running   0             72s
storage-provisioner                    1/1     Running   1 (24m ago)   25m
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl top pods
NAME                      CPU(cores)   MEMORY(bytes)
ai-app-5c86c57794-4mc82   1m           19Mi
ai-app-5c86c57794-c8xxg   1m           19Mi
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl top nodes
NAME       CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
minikube   206m         1%       814Mi           5%
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl autoscale deployment ai-app --cpu-percent=50 --min=2 -max=10
Flag --cpu-percent has been deprecated, Use --cpu with percentage or resource quantity format (e.g., '70%' for utilization or '500m' for milliCPU).
error: required flag(s) "max" not set
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl autoscale deployment ai-app --cpu-percent=50 --min=2 --max=10
Flag --cpu-percent has been deprecated, Use --cpu with percentage or resource quantity format (e.g., '70%' for utilization or '500m' for milliCPU).
horizontalpodautoscaler.autoscaling/ai-app autoscaled
(base) admin1@bflcomp20:~/ai-k8-app$ kubectl get hpa
NAME     REFERENCE            TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
ai-app   Deployment/ai-app    cpu: 1%/50%   2         10        2          19s
```

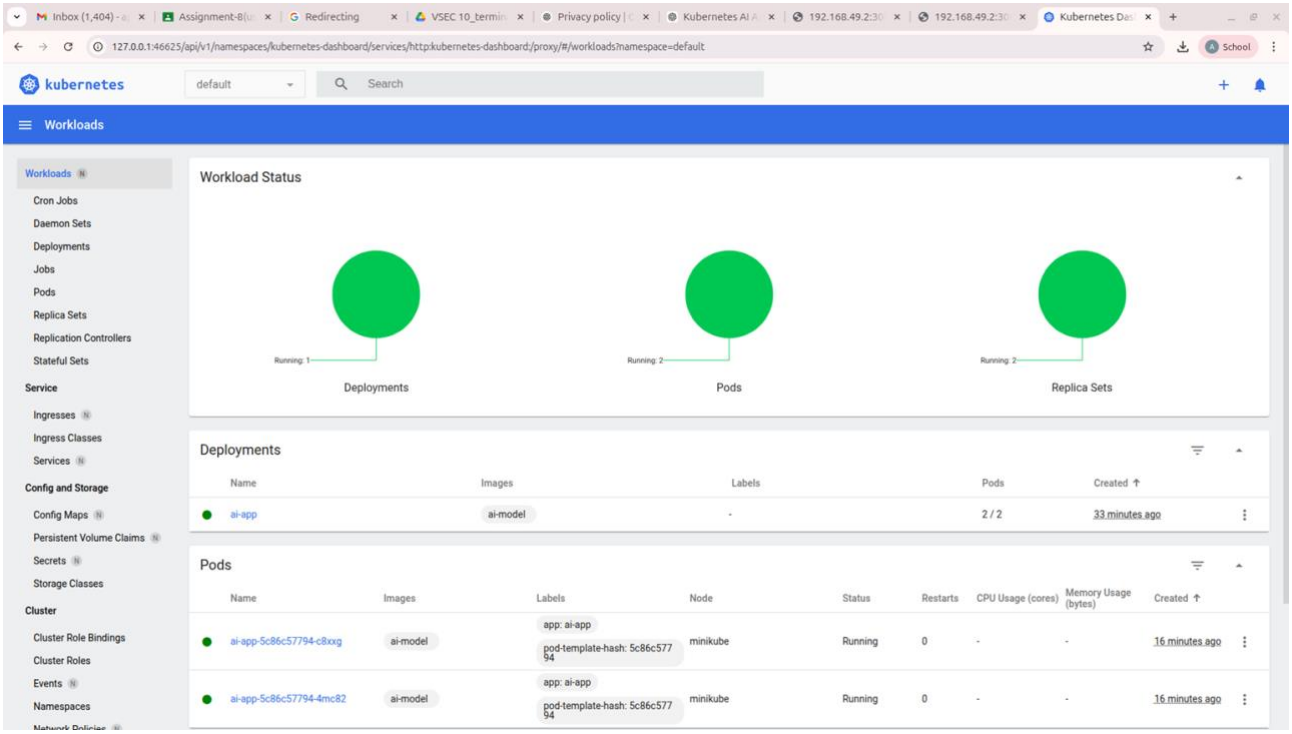
```
(base) admin1@bflcomp20:~/ai-k8-app$ minikube dashboard
🔌  Enabling dashboard ...
    ▪ Using image docker.io/kubernetesui/dashboard:v2.7.0
    ▪ Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
💡  Some dashboard features require the metrics-server addon. To enable all features please run:

        minikube addons enable metrics-server

🤔  Verifying dashboard health ...
🚀  Launching proxy ...
🤔  Verifying proxy health ...
🎉  Opening http://127.0.0.1:46625/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in your default browser...
Opening in existing browser session.
```




AI Model Running in Kubernetes


Inbox (1,404) -
Assignment-8(
Redirecting
VSEC 10_termin
Privacy policy |
Kubernetes AI A
192.168.49.2:30
192.168.49.2:30
Kubernetes Das
127.0.0.1:46625/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/#/workloads?namespace=default
School
kubernetes
default
Search
Workloads
Workloads
Cron Jobs
Daemon Sets
Deployments
Jobs
Pods
Replica Sets
Replication Controllers
Stateful Sets
Service
Ingresses
Ingress Classes
Services
Config and Storage
Config Maps
Persistent Volume Claims
Secrets
Storage Classes
Cluster
Cluster Role Bindings
Cluster Roles
Events
Namespaces
Workload Status
Running: 1
Deployments
Running: 2
Pods
Running: 2
Replica Sets
Deployments
Name
Images
Labels
Pods
Created
ai-app
ai-model
-
2 / 2
33 minutes ago
Pods
Name
Images
Labels
Node
Status
Restarts
CPU Usage (cores)
Memory Usage (bytes)
Created
ai-app-5c86c57794-c8xxg
ai-model
app: ai-app
pod-template-hash: 5c86c57794
minikube
Running
0
-
-
16 minutes ago
ai-app-5c86c57794-4mc82
ai-model
app: ai-app
pod-template-hash: 5c86c57794
minikube
Running
0
-
-
16 minutes ago